

Midterm Project: QA Implementation and Analysis

Group Name: CSE-2506M

Team Members:

- Medet Akhmetov
- Sultangali Kozhanov
- Alisher Temirkhan

1. System Description

NovaCart is a demo commerce platform built with a layered FastAPI backend, PostgreSQL persistence, Alembic migrations, and a React frontend served by the same application. The backend is organized into domain, application, infrastructure, and presentation layers, while the frontend supports integrated browser-based validation of the user journey.

The system supports product catalog browsing, user registration and login, cart management, coupon and shipping selection, checkout and order creation, product reviews, and admin operations such as product management, dashboard metrics, and order-status updates. Because the platform combines access control, transactional workflows, and persistence-backed business logic, it is a suitable candidate for a risk-based QA approach focused on the modules most likely to affect customer experience, data integrity, and release readiness.

2. Methodology

The project used a risk-based testing methodology. Each module was initially evaluated using probability of failure, business impact, and overall risk level. This produced a prioritized testing order in which Authentication, Shopping Cart, Checkout and Order Creation, Admin Product Management, Order Status Updates, and Database Migration and Persistence were treated as the most important areas for automation.

The automation strategy emphasized rapid coverage of high-risk workflows before lower-risk areas. Playwright was selected as the main automation framework because it supports both direct API testing and browser-based end-to-end validation in a single ecosystem, produces reusable reports, and fits cleanly into CI/CD execution. GitHub Actions was chosen as the CI/CD model, with saved logs, screenshots, HTML reports, JSON artifacts, and metric tables used as the evidence base for later reassessment.

The methodology also evolved across the midterm. After the initial strategy and coverage baseline were established, the preserved execution evidence was used to reassess risk, identify missing scenarios, expand automation coverage, collect measurable quality metrics, and compare the original plan against actual results.

3. Automation Implementation

The implemented automation spans three levels. Integration coverage is provided primarily by Playwright API tests under `tests/`, where the suite exercises FastAPI routes, authentication and authorization flows, cart behavior, checkout, admin operations, and persistence-backed scenarios. End-to-end coverage is provided by Playwright browser tests under `postman/`, which act as smoke and user-flow checks against the running application. Task 2 also introduced an explicit backend unit-test layer through Python tests focused on service-level logic such as pricing calculation, coupon removal, and shipping validation.

The automation expansion addressed several weaknesses identified in the earlier evidence. New scenarios were added for coupon removal, cart concurrency or repeated actions, database seeded-baseline validation, admin access control, and unauthenticated cart access in the UI. These additions improved the structural completeness of the suite by covering failure cases, edge conditions, invalid-user behavior, and race-sensitive flows that were underrepresented in the measured baseline.

The CI/CD implementation is defined in `github-actions-ci.yml`. The workflow installs backend dependencies, runs backend unit tests, applies Alembic migrations, starts FastAPI, performs health checks, executes Playwright API integration tests, executes browser smoke or E2E tests, uploads artifacts, and alerts on failure. This pipeline model supports the project's quality gates, which include minimum high-risk coverage, zero tolerated critical failures in key workflows, full regression pass requirements, and execution of unit, integration, and end-to-end checks in CI. However, the report distinguishes between implemented CI improvements and measured outcomes: the Task 2 pipeline structure was updated, but a full post-Task-2 rerun was not preserved in the current evidence set.

4. Results

The measured results show that the automation effort achieved broad coverage across the highest-risk workflows. Overall automation coverage across high-risk functions reached 92% (23/25). The preserved baseline recorded 100% measured coverage for Authentication, Checkout and Order Creation, and Admin Product Management, 86% for Shopping Cart, and 80% for Database Migration and Persistence. No selected high-risk module was below the 70% threshold, although coupon removal and explicit migration validation remained baseline gaps until they were implemented structurally in Task 2.

High-risk module	Measured baseline coverage
Authentication	100%
Shopping Cart	86%
Checkout and Order Creation	100%
Admin Product Management	100%
Database Migration and Persistence	80%

Defect evidence also confirmed that the original prioritization was meaningful. Authentication produced the highest preserved defect count with 6 failures and exceeded its expected range. Shopping Cart recorded 3 defects, Checkout and Order Creation recorded 2, Admin Product Management recorded 1, and Database Migration and Persistence recorded 1. Admin Dashboard, although not one of the original top high-

priority modules, emerged as the clearest medium-risk outlier with 2 recorded defects. The overall pattern shows that defect concentration remained strongest in business-critical areas, but the evidence also showed that some failures were caused by automation defects rather than confirmed product logic problems.

The execution metrics show strong efficiency. The full preserved API suite completed in 1.955 seconds, and the slowest module run was System/Admin at 1.195 seconds. This means runtime was never the limiting factor for CI use. Stability findings showed 0 flaky tests across the preserved artifacts. The main reliability problem was not intermittent behavior; it was deterministic failure caused by product mismatches, unstable setup, and Playwright fixture misuse. The preserved full-suite baseline was 37 passed, 20 failed, 54 skipped, and 0 flaky, which means the suite was broad enough to discover issues but not yet stable enough to act as a clean release gate.

Preserved execution metric	Value
Full suite passed	37
Full suite failed	20
Full suite skipped	54
Full suite flaky	0
Slowest module runtime	1.195 sec
Full suite runtime	1.955 sec

The risk reassessment reinforced these results. Authentication, Shopping Cart, and Checkout and Order Creation remained critical because they combined high business impact with persistent failures and reduced detectability. Admin Product Management and Database Migration and Persistence remained high risk because they still contained either automation instability or incomplete verification in areas that affect catalog integrity and data consistency. The metrics therefore support a mixed conclusion: coverage breadth and execution speed were strong, but regression trustworthiness remained limited.

5. Discussion

The strongest result of the midterm work is that the original risk-based prioritization was mostly correct. The modules identified early as the highest priority were the same modules that later showed the highest concentration of meaningful failures, coverage concerns, or confidence gaps. In that sense, the planning logic succeeded. The project did not suffer from testing the wrong areas; it suffered from the difference between broad automation presence and fully reliable regression quality.

Several assumptions were weaker than expected. Seeded registration and authentication flows were assumed to be stable enough to support downstream suite setup, but registration failures cascaded into cart and order scenarios. Broad coverage was assumed to imply that the suite was close to a trustworthy regression oracle, but detectability was reduced by harness problems such as Playwright fixture misuse. Operational migration handling was also assumed to be enough for persistence confidence, but the lack of direct migration validation left a clear gap in a high-risk area.

The results also showed missing scenarios and design inefficiencies. Coupon removal and explicit migration validation were missing from the measured baseline. Invalid-user behavior, access-control expansion, and concurrency-sensitive cart behavior were also underrepresented before the automation expansion. On the design side, shared setup

dependencies caused failures to cascade across modules, fixture misuse created false negatives, and reporter output-folder clashes created avoidable artifact-management risk. These were not just minor implementation details; they materially reduced the practical value of the automation as a release signal.

The next phase should focus on converting structural improvements into verified outcomes. The full pipeline should be rerun against the updated suite, fixture design should be cleaned so failures remain isolated to the feature under test, and the newly added Task 2 scenarios should be reflected in fresh measured metrics. If the project is extended further, adding dedicated linting or static-analysis evidence would also strengthen CI completeness and align the automation more closely with the defined quality gates.

Conclusion

The midterm evidence shows that NovaCart's QA strategy was directionally correct and technically productive. Risk-based prioritization led to broad automation coverage of the most business-critical workflows, and the resulting evidence base is strong enough to support meaningful analysis of quality, coverage, runtime, and defect concentration.

At the same time, the evidence also shows that broad automation alone is not enough. The preserved baseline still contains critical-path failures, incomplete scenario coverage, and automation-design weaknesses that prevent the suite from functioning as a fully trusted release gate. The most accurate overall conclusion is therefore balanced: the project built a strong foundation for QA automation, but additional cleanup and rerun validation are still required before the implemented strategy can be considered fully realized.

Evidence References

- docs/risk-assessment.md
- docs/QA_Test_Strategy_Assignment2.md
- docs/task1-risk-refinement.md
- docs/task2-automation-expansion.md
- docs/task3-metrics-analysis.md
- docs/task4-comparative-analysis.md
- github-actions-ci.yml
- evidence/AUTOMATION_COVERAGE_TABLE.md
- evidence/DEFECTS_RISK_COMPARISON_TABLE.md
- evidence/EXECUTION_TIME_TABLE.md
- evidence/QUALITY_GATE_TABLE.md
- evidence/
- logs/